

CSC 330 Programming Languages

Program #5

Prolog

Directions:

You are to create a parser in Prolog. No this will NOT require ANTLR or any other libraries, just plain Prolog in a single .pl file. The included file "src.txt" has the lines of code you will need to be able to parse. On the SWI terminal, you will invoke a custom predicate called "parse_file" (you will create this predicate), and it will open the src.txt file, going through it line-by-line to produce output.

Here is the grammar for the parser you are creating. You will need to convert this into Prolog code.

```
; convert these into Prolog rules
unit ::= assignment
      | print_call

assignment ::= SET IDENT EQUALS math_exp

print_call ::= PRINT math_exp

math_exp ::= num_or_ident ADD math_exp
          | num_or_ident MUL math_exp
          | num_or_ident

num_or_ident ::= NUM
              | IDENT

; convert these into Prolog facts
PRINT  ::= 'print'
SET    ::= 'set'
EQUALS ::= '='
ADD    ::= '+'
MUL    ::= '*'

; convert these into Prolog rules
NUM    ::= [0-9]+
IDENT  ::= [a-zA-Z]+
```

In the src.txt input file, a single space can be used as a token delimiter. To keep things simple, each line of code should be processed separately as you read the file line-by-line. This makes it so you don't have to have recursion for the "unit" non-terminal.

Since Prolog is all about predicates, each of the non-terminals can be a predicate and you can use pattern matching to match the different rules for a non-terminal. The terminal tokens can be simple Prolog facts that match the corresponding text. NUM and IDENT will need to be rules since they are more complicated. Do note that IDENT is only composed of letters. This should make tokenization is bit easier.

Parsing process:

I strongly suggest getting one small thing working at a time and creating your own test file. Get one small thing working and only move on to the next after you have tested it. Do NOT try to get everything working at once.

The process of your logic should be as follows:

1. First get it to read from a file. I recommend saving a copy of the given src.txt file and then changing it to be a lot simpler so you can test.
2. Prolog should read this file in line by line and send each line to a tokenize predicate, which should give back a list of strings. For example, if the line of code is:

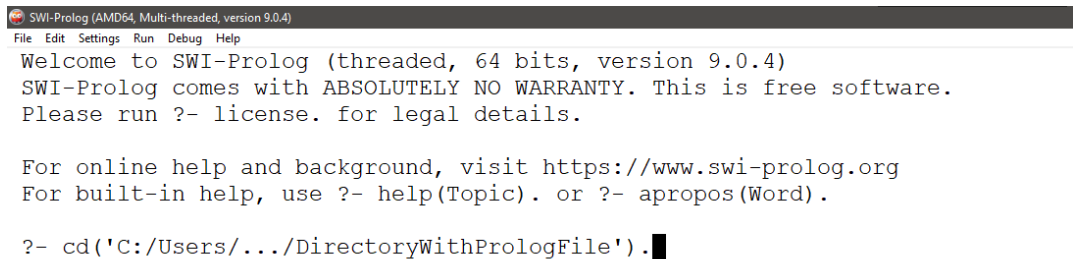
```
myVariable = 3
```

Your tokenization should result in the list `["myVariable", "=", "3"]`. Note that you can use a single space as a token delimiter.
3. Next, we should send the token list to a "unit" predicate. This is where the grammar kicks in. Unit will lead to "assignment" or "print_call" predicate and so on, according to the grammar.
4. When we see a math_exp expression, we should manually evaluate the expression. This can be done by making math_exp recursive (as is shown in the grammar) and having Prolog perform the math (using "is" for evaluation). We only need to do addition and multiplication here. Notice how the grammar uses right recursion for math_exp. This isn't mathematically correct however getting left recursion to work in Prolog is sometimes tricky. So to make things easier, you can use right recursion like the grammar has. If we stick to only + and *, we will still get the right result. Just don't mix the operators, since we also have no concept of precedence here! We could fix both of these issues in Prolog, but for the sake of not making this assignment any more difficult than it already is, we can just leave math_exp as right associative and not mix + and * operators in a single line. Feel free to make it left associative in Prolog if you find that easier (the final result will be the same).
5. When we see an assignment expression, we should evaluate the rvalue expression. We should then take that rvalue and assign a global variable to it, using the name of the identifier. SWI Prolog has a built-in predicate called "nb_setval" that will help with this. This will allow us to actually set the value of the specified variable. If the rvalue expression has variables in it, we need to use "nb_current" to retrieve that value.
6. When we see a print expression, we should evaluate the rvalue expression and write that to the output. Just like with assignment, if the rvalue expression has variables in it, we need to use "nb_current" to retrieve that value.
7. When we are finished, we should go to the next line in the file and do it all over again. This continues until we reach "end_of_file" (which is a special atom in Prolog for detecting this).

Hints:

Workflow

When you open SWI-Prolog, remember to use the `cd` predicate to navigate to your prolog file:



```
SWI-Prolog (AMD64, Multi-threaded, version 9.0.4)
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 9.0.4)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- cd('C:/Users/.../DirectoryWithPrologFile').
```

You must use either forward slash `/` or escape the backslash `\\` in your path (a single backslash will result in error).

Then use `['filename']` (or `consult('filename')`) to load (or reload) your file:

```
?- ['parser'].
?- consult('parser').
```

The single quotes can be omitted if you wish. Both `[]` and `consult` are equivalent so pick whichever way you like. In either case, you don't need to include the file extension. After loading (or reloading your file if you modify it), you can use any predicates you have defined in the file.

Remember that you can use the up/down arrows to scroll through your history like in a normal terminal. Also, remember that the command you enter must end with a period (similar to semicolon in Java).

Use Cut

The cut operator `!` is your friend! Remember that this operator will prevent Prolog from backtracking past that point. A lot of times Prolog will fail to match a rule because of either a syntax error in the `src.txt` or because of some mistake in your code, and because of this fail, Prolog will jump to another predicate to try a different route. This will cause predicates to be used on input that it wasn't designed for and could cause a crash in your program. While you are writing this program, this behaviour can make troubleshooting take a lot longer and a simple mistake on your part can cause hours of debugging. Cut prevents this by telling Prolog that past this point, there is no return! Without this operator, your program could crash on a syntax error in the `src` instead of just returning `"false"`.

Trace

If you are having trouble following the logic in your program to see why something isn't working, you might want to turn on tracing. Just type `trace.` to turn this on:

```
?- trace.
true.

[trace] ?-
```

The prompt now has the word `"trace"` before it. Any predicates you run in this mode will cause Prolog to go through each step in the unification process. Essentially it is like a debugger. Hit `<enter>` to progress through the trace.

If you type something other than enter and trace doesn't understand it, it will tell you to type "h" for help. If we then type "h", we get a list of commands we can use. Typing "a" will abort the trace. Typing "s" will skip over the resolution of the predicate you are about to resolve. Feel free to try the different commands to see what they do. This will help you greatly in debugging your code.

To turn tracing off, just type notrace. The trace predicate also turns on debug mode. So, you will also need to type nodebug. to turn that off as well.

```
[trace]  ?- notrace.  
true.
```

```
[debug]  ?- nodebug.  
true.
```

```
?-
```

Sample output:

Printing in color is optional but I think you will find the output much easier to read quickly if colors are added. The source line should be printed first in **green** (i.e. set color to "\033[32m"). This should be followed by an arrow --> printed in **yellow** (i.e. "\033[33m"). Then, if the statement is an assignment, output should be of the form VARIABLE = FINAL_VALUE. If the statement is a print, output should just show the final value being printed. Both of these should be shown in **purple** (i.e. "\033[35m").

For the given "src.txt" input file, the final output would be the following:

```
set alpha = 5 --> alpha = 5  
set beta = alpha --> beta = 5  
set gamma = 42 * beta --> gamma = 210  
set delta = 1 + 2 + 3 + 4 + 5 --> delta = 15  
set epsilon = 2 * 4 * beta --> epsilon = 40  
print 1 + 2 --> 3  
print 3 * 5 * 2 --> 30  
print alpha --> 5  
print beta + 1 --> 6  
print gamma * 3 --> 630  
print delta --> 15  
print 4 + epsilon + 2 --> 46
```

The output must look exactly like this either in color or without colors added.

Important Note:

For this assignment, you are NOT to use any Prolog libraries. You are also not allowed to use any operators that are not covered in the lecture notes. This includes the "->" operator. Any built in predicates are fair game, though.

For submission:

Submit only your Prolog file. Name the file "parser.pl" and submit on Canvas. Remember that it should have a predicate called "parse_file" that doesn't accept parameters and just parses the src.txt file that's in the same directory. This parse_file predicate may invoke a version of the predicate that takes in a file (refer to the template to see what I mean).

Rubric:

#	ITEM	POINTS
1	Assignment works and saves to variables	15
2	Printing works and recalls variables	15
3	Math expressions work	10
4	Grammar is represented as predicates	10
5	Properly reads from source code file	5
6	Output is correct	5
	TOTAL	60

Penalties

#	PENALTIES	POINTS
1	Doesn't compile	-50%
2	Doesn't execute once compiled (i.e. it crashes)	-25%
3	Late up to 1 day	-25%
4	Late up to 2 days	-50%
5	Late after 2 days	-100%